

CS336 Assignment 2 实验报告

linyin

2026 年 7 月 5 日

1 Benchmarking Script

1.1 实验设置

本实验使用 NVIDIA GeForce RTX 4090，对 small 和 medium 模型在不同 context length 下的一次完整训练步骤进行性能测试。实验配置为 batch size 4，首先执行 5 次预热，随后进行 10 次正式测量。

1.2 带 warm-up 实验结果

实验在一张 NVIDIA GeForce RTX 4090 (24GB 显存) 上进行。模型使用 batch size 4，首先执行 5 次预热，随后进行 10 次正式测量。测量结果如表 1 所示，其中 “-” 表示由于显存不足而未能获得结果。

表 1: 不同模型规模和上下文长度下的性能测试结果

模型	Context length	Forward (ms)	Backward (ms)	Optimizer (ms)	运行状态
Small	256	28.841 ± 0.483	56.182 ± 2.504	13.002 ± 0.360	成功
Small	512	28.189 ± 0.926	55.628 ± 1.992	12.797 ± 0.285	成功
Small	1024	72.593 ± 0.229	150.140 ± 0.151	12.867 ± 0.164	成功
Medium	256	57.477 ± 0.744	109.311 ± 3.473	40.147 ± 0.380	成功
Medium	512	75.015 ± 0.336	155.394 ± 0.389	39.880 ± 0.359	成功
Medium	1024	-	-	-	OOM

在当前单卡 RTX 4090 24GB 显存限制下，small 模型能够运行到 context length 1024，而 medium 模型在 context length 1024 时发生显存不足，因此 medium 模型最大成功运行的上下文长度为 512。由于作业要求选择大于 128 的 2 的幂次上下文长度，并尽量选择显存能容纳的最大长度，因此本文报告 small 模型的 256、512、1024 三组结果，以及 medium 模型的 256、512 两组成功结果，并将 medium context length 1024 记为 OOM。

从结果可以看到，反向传播始终是耗时最高的阶段，通常约为前向传播的两倍左右。这是因为反向传播不仅需要计算梯度，还需要沿计算图反向执行大量矩阵乘法和逐元素操作。Optimizer 阶段的耗时主要与模型参数量有关，因此在同一模型规模下，context length 从 256 增加到 1024 时，small 模型的 optimizer 时间基本保持在约 13 ms；medium 模型的 optimizer 时间也基本保持在约 40 ms。

对于 small 模型，context length 从 256 增加到 512 时，forward 和 backward 时间变化不大；但增加到 1024 时，forward 从约 28 ms 增加到约 73 ms，backward 从约 56 ms 增加到约 150 ms。这说明在较短序列长度下，GPU kernel launch、调度开销和 GPU 利用率等因素可能掩盖序列长度增长带来的开销；而当 context length 增加到 1024 后，attention 中与序列长度平方相关的计算和显存访问开销开始明显体现出来。

对于 medium 模型，context length 从 256 增加到 512 时，forward 从约 57 ms 增加到约 75 ms，backward 从约 109 ms 增加到约 155 ms，增长趋势更加明显。整体来看，模型规模和上下文长度都会显著影响训练速度，而显存限制会很快成为更大模型和更长上下文训练的主要瓶颈。

1.3 改变 Warm-up 次数的实验结果

1.3.1 Warm-up = 0

在不进行预热的情况下，各模型的测量结果如表 2 所示。实验配置保持为 batch size 4、context length 512，并正式测量 10 次。

表 2: 不进行预热时的性能测试结果

模型	Forward (ms)	Backward (ms)	Optimizer (ms)
Small	53.807 $\pm$ 87.837	67.246 $\pm$ 38.523	17.321 $\pm$ 14.372
Medium	125.609 $\pm$ 153.896	167.544 $\pm$ 40.169	43.418 $\pm$ 11.581
Large	197.071 $\pm$ 84.114	350.336 $\pm$ 33.491	—

与 5 次预热的结果相比，未预热时各阶段的平均耗时普遍更高，标准差也显著增大。其中 Small 和 Medium 模型的 Forward 标准差甚至超过了平均值，说明 10 次测量中存在耗时特别高的初始迭代。

这是因为第一次运行可能包含 CUDA 上下文和动态库初始化、GPU 显存首次分配、内核加载以及 AdamW 状态创建等一次性开销。随着模型重复运行，这些初始化工作不再出现，因此后续迭代明显更快，最终造成较大的测量波动。

1.3.2 Warm-up = 1

执行 1 次预热后的完整训练步骤结果如表 3 所示。实验配置保持为 batch size 4、context length 512，并正式测量 10 次。

表 3: 执行 1 次预热时的完整训练步骤性能

模型	Forward (ms)	Backward (ms)	Optimizer (ms)
Small	24.930 ± 1.568	55.247 ± 1.691	12.596 ± 0.268
Medium	74.251 ± 0.254	154.445 ± 0.712	39.613 ± 0.383

Small 模型完整训练步骤的各阶段耗时之和为：

$$24.930 + 55.247 + 12.596 = 92.773 \text{ ms.}$$

Medium 模型完整训练步骤的各阶段耗时之和为：

$$74.251 + 154.445 + 39.613 = 268.309 \text{ ms.}$$

另外，在 Medium 模型上使用仅包含前向和反向传播的模式进行了一次对照实验，结果如表 4 所示。

表 4: Medium 模型在不同运行模式下的耗时比较

运行模式	Forward (ms)	Backward (ms)
Full step	74.251 ± 0.254	154.445 ± 0.712
Forward + backward	74.617 ± 0.300	154.633 ± 0.517

不进行预热时，测量结果的平均值和标准差都明显增大，例如 Small 模型的 Forward 由 5 次预热后的稳定结果变为 $53.807 \pm 87.837 \text{ ms}$ ，说明初始迭代包含 CUDA 初始化、显存分配、内核加载和 AdamW 状态创建等一次性开销。执行 1 次预热后，大部分一次性开销已经被排除，结果接近 5 次预热，但 Small 模型的波动仍然更大。1~2 次预热可能仍不足以让 GPU 频率、缓存、内存分配器和各类延迟初始化完全稳定，因此使用 5 次预热能得到更加可靠的稳态性能结果。

1.4 nsys 性能分析

为了进一步分析模型前向传播、反向传播和优化器更新中具体的 CUDA kernel 开销，我使用 NVIDIA Nsight Systems 进行 profile。由于已经在代码中手动添加了 NVTX 标记，因此本实验没有启用 PyTorch 自动函数级 trace，以避免大量额外 trace 事件带来的 profiler 开销。

一个示例的 nsys 命令如下，实际批量运行命令放在 `run_all_nsys.sh` 中。

```
nsys profile \
  --trace=cuda,nvtx,cudnn,cublas,osrt \
  --sample=none \
```

```

--cpuctxsw=none \
--output=reports/small_ctx512_full_step_light \
--force-overwrite=true \
-- python benchmark_nvtx.py \
--model-size small \
--context-length 512 \
--mode full_step \
--warmup-steps 5 \
--measurement-steps 10

```

其中, `--trace=cuda,nvtx,cudnn,cublas,osrt` 表示采集 CUDA、NVTX、cuDNN、cuBLAS 以及操作系统运行库相关事件。`--sample=none` 用于关闭 CPU IP/backtrace sampling, `--cpuctxsw=none` 用于关闭线程上下文切换采集。这样可以避免在当前服务器环境中由于权限限制产生 CPU 采样错误, 同时也能减少 profiler 的额外开销。

在 benchmark 代码中, 我手动添加了如下 NVTX 区间:

- `my_train_step`: 一次完整训练步骤;
- `my_forward`: 前向传播;
- `my_cross_loss`: 交叉熵损失计算;
- `my_backward`: 反向传播;
- `my_optimizer_step`: AdamW 参数更新;
- `my_attention_scores_matmul, my_attention_softmax, my_attention_value_matmul`: self-attention 内部的主要子步骤。

在 Nsight Systems GUI 中, 我通过对对应的 NVTX 区间使用 Apply to Filter, 再查看 Stats System View 中的 CUDA GPU Kernel Summary, 从而统计指定阶段内部的 CUDA kernel 累计时间。

1.4.1 实验结果分析

(a) 前向传播总时间 根据 Python 标准库计时结果, small 模型在 context length 为 256、512、1024 时的前向传播平均耗时分别为:

28.841 ms, 28.189 ms, 72.593 ms.

在去掉 PyTorch 自动函数级 trace 后, Nsight Systems 中 `my_forward` NVTX 区间显示的耗时与 Python 标准库测量结果基本一致。此前开启 `--pytorch=functions-trace, autograd-sha` 时, Nsight 中 forward 区间时间明显偏大, 这是因为 PyTorch 函数级 tracing 会插入大

量额外 NVTX 事件和记录开销，导致 profiler 本身显著影响运行时间。因此，后续分析主要使用手动添加的 NVTX 标记，而不启用 PyTorch 自动函数级 trace。

对于 small 模型，context length 从 256 增加到 512 时，forward 时间几乎没有增加；但当 context length 增加到 1024 时，forward 时间明显增加到约 72.6 ms。这可能是因为在较短序列长度下，GPU 调度开销、kernel launch 开销以及 GPU 利用率等因素占比较高，而当序列长度达到 1024 后，attention 中随序列长度平方增长的计算和显存访问开销开始占据主导。

对于 medium 模型，context length 为 256 和 512 时，前向传播时间分别为：

57.477 ms, 75.015 ms.

medium 模型在 context length 1024 时由于显存不足无法完成完整训练步骤。

(b) 前向传播中累计 GPU 时间最高的 CUDA kernel 为了回答该问题，我在 Nsight Systems GUI 中对单次 my_forward NVTX 区间应用 filter，然后查看 CUDA GPU Kernel Summary，并按照 Total Time 降序排序。

以 small 模型、context length 256 为例，前向传播中累计 GPU 时间最高的 CUDA kernel 是：

ampere_sgemm_128x64_tn

该 kernel 在一次 forward pass 中被调用 85 次，累计 GPU 执行时间为 7.238 ms，占该 forward 区间内 GPU kernel 总时间的 69.7%。该 kernel 是 Ampere 架构上的 SGEMM 矩阵乘法 kernel，即 FP32 general matrix multiplication。这个结果符合预期，因为 Transformer 的主要计算量来自线性层、MLP 以及 attention 中的矩阵乘法。

需要注意的是，这里的 7.238 ms 并不是整个 forward pass 的 wall-clock 时间，而只是某一种 CUDA kernel 的累计 GPU 执行时间。一次 my_forward NVTX 区间的总耗时会包含 CPU 侧 PyTorch dispatch、CUDA API launch、cuBLAS 调用开销、kernel 之间的空隙以及 profiler 本身的记录开销；而 CUDA GPU Kernel Summary 只统计 GPU kernel 实际在 GPU 上执行的时间。因此，forward NVTX 区间的总时间通常会大于单个 kernel 的累计执行时间。

在同时执行 forward 和 backward 的 profile 中，排名靠前的 kernel 仍然主要是 SGEMM/GEMM 类矩阵乘法 kernel。这说明无论是前向传播还是反向传播，矩阵乘法都是 Transformer 计算中的主要 GPU 时间消耗来源。

(c) 除矩阵乘法以外的显著 CUDA kernel 在 my_forward 区间内，除了 SGEMM/GEMM 矩阵乘法 kernel 之外，还可以观察到一些非矩阵乘法 kernel 占据非忽略的 CUDA runtime，例如：

- `at::native::elementwise_kernel;`

- `at::native::vectorized_elementwise_kernel`;
- reduction 相关 kernel;
- softmax 相关 kernel;
- cuBLASLt 的 split-K reduction kernel。

这些 kernel 对应逐元素操作、归一化、激活函数、attention softmax、张量 reduction 或 cuBLAS 内部归约等操作。虽然这些操作的 FLOPs 远少于矩阵乘法，但它们往往受到显存带宽、kernel launch 开销和访存模式的影响，因此仍然会占据一定的运行时间。

(d) 完整训练步骤中矩阵乘法占比的变化 完整训练步骤包括 forward、loss、backward 和 optimizer step。与仅进行 forward 相比，加入 backward 后会额外引入大量梯度计算相关的矩阵乘法 kernel，因此 GEMM 类 kernel 仍然是主要的 GPU 时间来源。不过，完整训练步骤还会引入更多逐元素 kernel、reduction kernel 以及优化器更新 kernel。

AdamW 的 optimizer step 与 forward/backward 不同，它基本不进行大型矩阵乘法，而是对参数、动量和二阶矩估计进行逐元素更新。因此，在 `my_optimizer_step` 区间内，主要出现的是 `multi_tensor_apply_kernel`、`elementwise_kernel` 和 `vectorized_elementwise_kernel` 等逐元素更新相关 kernel。也就是说，完整训练步骤中矩阵乘法的绝对耗时会增加，但由于 backward 和 optimizer 引入了更多其他类型 kernel，矩阵乘法在总时间中的占比不一定会像 forward-only 中那么高。

(e) self-attention 中 softmax 与矩阵乘法的比较 在 self-attention 层内部，我分别对 attention score 矩阵乘法、softmax 和 value 矩阵乘法添加了 NVTX 标记：

- `my_attention_scores_matmul`：计算 QK^T ；
- `my_attention_softmax`：对 attention scores 计算 softmax；
- `my_attention_value_matmul`：计算 $\text{softmax}(QK^T)V$ 。

理论上，attention 中两次矩阵乘法的 FLOPs 远大于 softmax。然而在实际 profile 中，softmax 的运行时间并不会按照 FLOPs 比例变得极小。这是因为矩阵乘法可以由 cuBLAS 或 Tensor Core 高度优化，计算密集度较高；而 softmax 涉及读取 attention score、求最大值、指数运算、求和、归一化以及写回结果，整体更容易受到显存带宽、访存模式和 kernel launch 开销的影响。因此，softmax 的 FLOPs 虽然远少于矩阵乘法，但它仍然可能占据不可忽略的 CUDA runtime。